Git 与仓库工具

文件工具和 git 元数据

Git 与仓库工具

- 背景
- 目标与非目标
- 本模块在仓库中的位置
- 接入 agent 循环
- 核心数据结构
  - GitPaths 的判定
  - 变更队列
  - 路径约束
- 方案
- 备选方案
- 横切关注点
- 手搓实现笔记

Git 与仓库工具

背景

上游 coding-agent 没有名为 `git` 的工具。改仓库用 `packages/coding-agent/src/core/tools/` 里的 `read`、`write`、`edit`、`grep`、`find`、`ls`、`bash`。分支显示用 `core/footer-data-provider.ts` 的 `findGitPaths`，命令是 `git --no-optional-locks symbolic-ref --quiet --short HEAD`。`utils/git.ts` 的 `parseGitUrl` 和 `PackageManager.installGit` 只用于安装 Pi 包，不用于用户仓库。本仓库不实现 `installGit`。

`@my/tools` 把这些工具做成 `AgentTool`。git 元数据是只读观察器，放在 `git-paths.ts`，不进入模型的工具表。

目标与非目标

目标

- `createDefaultTools(cwd)` 返回固定的 7 个工具，名字与上游一致。
- worktree 和普通仓库都能解析出 `HEAD` 路径。

- 对同一 cwd 的 write/edit 不会并行执行。

非目标

- 不提供 `git_status`、`git_commit`、`git_worktree`。模型要跑 git 就调用 `bash`。
- 不自动 `git worktree add`。上游 durable 示例里的 worktree 是测试夹具的 cwd，不是工具。
- 不解析 packfile、reftable 或 `HEAD` 文件内容来猜分支名。

本模块在仓库中的位置

```
packages/tools/
  package.json
  tsconfig.json
  src/index.ts                 # createDefaultTools
  src/truncate.ts              # 上游 core/tools/truncate.ts
  src/file-mutation-queue.ts   # 上游 core/tools/file-mutation-queue.ts
  src/read.ts
  src/write.ts
  src/edit.ts
  src/grep.ts
  src/find.ts
  src/ls.ts
  src/bash.ts
  src/git-paths.ts             # 上游 footer-data-provider.ts 的 findGitPaths
  test/truncate.test.ts
  test/mutation-queue.test.ts
  test/git-paths.test.ts
  test/edit.test.ts
```

文件	职责
`truncate.ts`	把工具输出限制在 2000 行或 50 KiB
`file-mutation-queue.ts`	按 cwd 串行化写操作
`read.ts` / `write.ts` / `edit.ts`	文件读写。edit 是精确字符串替换
`grep.ts` / `find.ts` / `ls.ts`	搜索和列目录，用 Node API，不调用 `rg` 二进制
`bash.ts`	`child_process.spawn`，cwd 固定，`executionMode: "sequential"`
`git-paths.ts`	找 `.git`，问 git 当前分支名
`index.ts`	组装工具数组。不在这里探测分支

依赖：import `@my/agent-core` 的 `AgentTool` 和 `AgentToolResult`。禁止 import session、mcp、tui。

磁盘：工具读写的是调用方传入的 cwd 下的项目文件，不是 `~/.my-agent`。`git-paths.ts` 只读 `.git` 和 `HEAD`。分支名不写入 JSONL。

接入 agent 循环

`apps/cli/src/wire.ts` 在构造 `Agent` 之前：

```
const tools = createDefaultTools(cwd);
agent.state.tools = tools;
```

之后工具表变化由 agent-core 的 `declareToolChanges` 宣布。工具 `execute` 抛错会被循环收成 `isError: true`。不要在工具里捕获之后再返回成功文本。

`resolveBranch` 不在循环里调用。M6 的页脚每 500ms 去抖后调用它。M4 的测试直接调用函数。

核心数据结构

```
export const DEFAULT_MAX_LINES = 2000;
export const DEFAULT_MAX_BYTES = 50 * 1024;

export interface TruncationResult {
  content: string;
  truncated: boolean;
  originalLines: number;
  originalBytes: number;
}

export interface GitPaths {
  repoDir: string;
  commonGitDir: string;
  headPath: string;
}

export interface ReadParams { path: string; offset?: number; limit?: number }
export interface ReadDetails { path: string; truncated: boolean }

export interface WriteParams { path: string; content: string }
export interface WriteDetails { path: string; bytes: number }

export interface EditParams { path: string; oldText: string; newText: string }
export interface EditDetails { path: string; replacements: number }

export interface BashParams { command: string }
export interface BashDetails { exitCode: number | null; truncated: boolean }

export interface GrepParams { pattern: string; path?: string }
export interface FindParams { pattern: string; path?: string }
export interface LsParams { path?: string }
```

`truncateHead` 从开头保留，超出时在末尾加一行 `\n[truncated]\n`。字节数按 UTF-8。先按行切，再按字节切。两个上限都生效，先触发的那个停。

GitPaths 的判定

`findGitPaths(cwd)` 从 `path.resolve(cwd)` 向上：

1. 没有 `.git`：去父目录。父目录等于自己时返回 `null`。
2. `.git` 是目录，且 `<git>/HEAD` 存在：`repoDir` 是该目录的父目录的工作树，也就是含 `.git` 的那一层；`commonGitDir` 是 `.git` 目录；`headPath` 是 `HEAD`。
3. `.git` 是文件，第一行匹配 `^gitdir: (.+)$`：把捕获组按 `.git` 文件所在目录做 `path.resolve`。该目录下必须有 `HEAD`。若还有 `commondir` 文件，`commonGitDir` 是相对这个 gitDir 解析后的路径，否则 `commonGitDir` 就是这个 gitDir。`repoDir` 是含 `.git` 文件的工作树根。

4. stat 抛错或 `HEAD` 不存在：返回 `null`。不要向上继续，避免把更高层的另一个仓库当成当前仓库。这是手搓默认，写进测试：坏的 `.git` 文件使整个查找失败。

```
export function findGitPaths(cwd: string): GitPaths | null
export function resolveBranch(repoDir: string, exec?: GitExec): Promise<string | null>

export type GitExec = (repoDir: string) => Promise<{ status: number; stdout: string }>;
```

默认 `exec` 运行：

```
git --no-optional-locks symbolic-ref --quiet --short HEAD
```

`status === 0` 且 `stdout.trim()` 非空时返回分支名。否则返回 `null`。调用方在 `findGitPaths` 非空且分支为 `null` 时显示字符串 `"detached"`。测试注入 `exec`，不依赖本机仓库。

变更队列

```
export class FileMutationQueue {
  constructor(cwd: string);
  run<T>(fn: () => Promise<T>): Promise<T>;
}
```

同一个 `FileMutationQueue` 实例上的 `run` 串成链。`createDefaultTools` 为每个 cwd 建一个队列，write 和 edit 的 `execute` 包在 `queue.run` 里。read、grep、find、ls、bash 不使用队列。bash 用 `executionMode: "sequential"`，因此只要这一批里有 bash，agent-core 会把整批改成串行。这是相对上游的收紧。

路径约束

所有 `path` 参数先 `path.resolve(cwd, input)`。结果必须等于 cwd 或以 `cwd + path.sep` 开头。否则抛 `Error("path escapes cwd")`。不要跟随符号链接走出 cwd 之后再写。读可以 `realpath`，若 realpath 走出 cwd，同样抛错。

edit 的替换次数不是 1 时抛错，文件不写。`oldText === ""` 抛错。write 使用 `fs.writeFile`，父目录不存在则 `mkdir` 递归创建。

bash 不走 shell 字符串拼接以外的检查：`spawn(shell, ["-c", command], { cwd, signal })`。`shell` 在 win32 上是 `powershell.exe`，否则是 `bash`。超时 120 秒，到点 kill 并抛错。stdout 和 stderr 合并后交给 `truncateHead`。退出码非 0 时抛 `Error`，消息里带退出码和截断后的输出，让循环把它变成 `isError`。

方案

`createDefaultTools` 的返回顺序固定为 read、write、edit、grep、find、ls、bash。`label` 用首字母大写的名字。`description` 各用一句话，测试不断言文案，只断言 `name`。

grep 用逐文件读取加 `RegExp`，默认忽略 `node_modules` 和 `.git` 目录。不要调用系统 `rg`，否则 CI 机器没有 ripgrep 时测试失败。find 按文件名的子串匹配，`pattern` 不是正则。ls 只列一层，不递归。

worktree 去重不在本包做。本包只导出：

```
export function shadowedContextFile(paths: GitPaths | null, worktreeContextBasename: string): string | undefined
```

当 `paths` 存在、`commonGitDir` 不是 `repoDir` 下的 `.git` 目录（说明是 linked worktree）、且 `repoDir` 位于 `dirname(commonGitDir)` 之内时，返回 `path.join(dirname(commonGitDir), worktreeContextBasename)`。否则 `undefined`。`wire.ts` 把这个返回值传给 `loadProjectContextFiles` 的 `shadowedPath`。basename 用 worktree 根上实际加载到的那个文件名。M4 测试只断言这个函数的返回路径，不断言 prompt。

备选方案

方案	放弃原因
libgit2	上游用 git 可执行文件。分支只是 UI 数据
解析 `HEAD` 文本	worktree 和 reftable 会分叉。`symbolic-ref` 失败即 detached
bash 与 edit 完全并行	上游允许。手搓把 bash 标成 sequential，避免测试成为时间竞赛

横切关注点

- **错误**：git 不在 `PATH` 时 `resolveBranch` 返回 `null`，不抛到进程外。工具错误变成 tool result。
- **并发**：`FileMutationQueue` 用一个 `Promise` 链，不用布尔锁。测试里两个 `run` 的开始时间必须满足第二个的 fn 在第一个 fn resolve 之后才开始。
- **持久化**：文件写入和 JSONL 的 tool result 不是一个事务。进程死在 write 之后、`message_end` 之前时，工作区已改、会话可能还没有结果。恢复时 M2 会补错误 tool result。不要在工具里回滚文件。
- **锁**：分支命令带 `--no-optional-locks`。不要为了显示分支运行 `git status`。

手搓实现笔记

按总览 M4 的 6 步。`git-paths.test.ts` 的 worktree 夹具手写文件：

```
repo/.git/HEAD                      # 内容 "ref: refs/heads/main"
repo/.git/worktrees/feat/HEAD
```

```
repo/.git/worktrees/feat/commondir     # 内容 "../.."
repo/worktrees/feat/.git               # 内容 "gitdir: <绝对或相对路径>"
```

这和上游 `resource-loader` 测试的骨架相同，不调用 `git worktree add`。